1.

Problem:

Given a linked list, return the **middle node**.
If even number of nodes, return the **second middle**.

Example:

1 → 2 → 3 → 4 → 5
Output: 3

1 → 2 → 3 → 4 → 5 → 6
Output: 4

Concept:

- slow = 1 step
- fast = 2 steps

2.

Problem:

Check whether a linked list contains a cycle (loop).

Example:

1 → 2 → 3 → 4 → 2 (cycle)
Output: true

1 → 2 → 3 → NULL
Output: false

Concept:

- If slow == fast → cycle exists

3.

Problem:

If a cycle exists, return the **node where cycle begins**.

Example:

1 → 2 → 3 → 4
↑ ↓
6 ← 5

Output: **3**